QA4.1 - Linguagens de Programação I: Questões de aula (QA) para a aula 1 de "Tipos de Dados"

Total de pontos **10/21** ?

Estas questões devem ser respondidas após o envio das questões QP, feitas em preparação para a aula 1 de "Nomes e Vinculação".

Idealmente, neste tempo você poderá:
* Responder individualmente as questões
* Discutir com o seu colega de classe as respostas escolhidas.
* Resubmeter a sua resposta.

A sua nota será a proporção dos acertos das questões. Por isso, responda com cuidado e tempo.

E-mail *

ramonsoares@discente.ufcat.edu.br

0 de 0 pontos

Nome *

RAMON SOARES MENDES DE MENESES LEITE ▾

Tipo de Submissão de QA *

○ Primeira submissão (individual)
● Segunda submissão (discutida com outro aluno)

Identificação de par em discussão

0 de 0 pontos

Procure na descrição da aula (Google Classroom) qual é a pessoa alocada para discutir com você essa questão.

Aluno com quem a questão foi discutida *

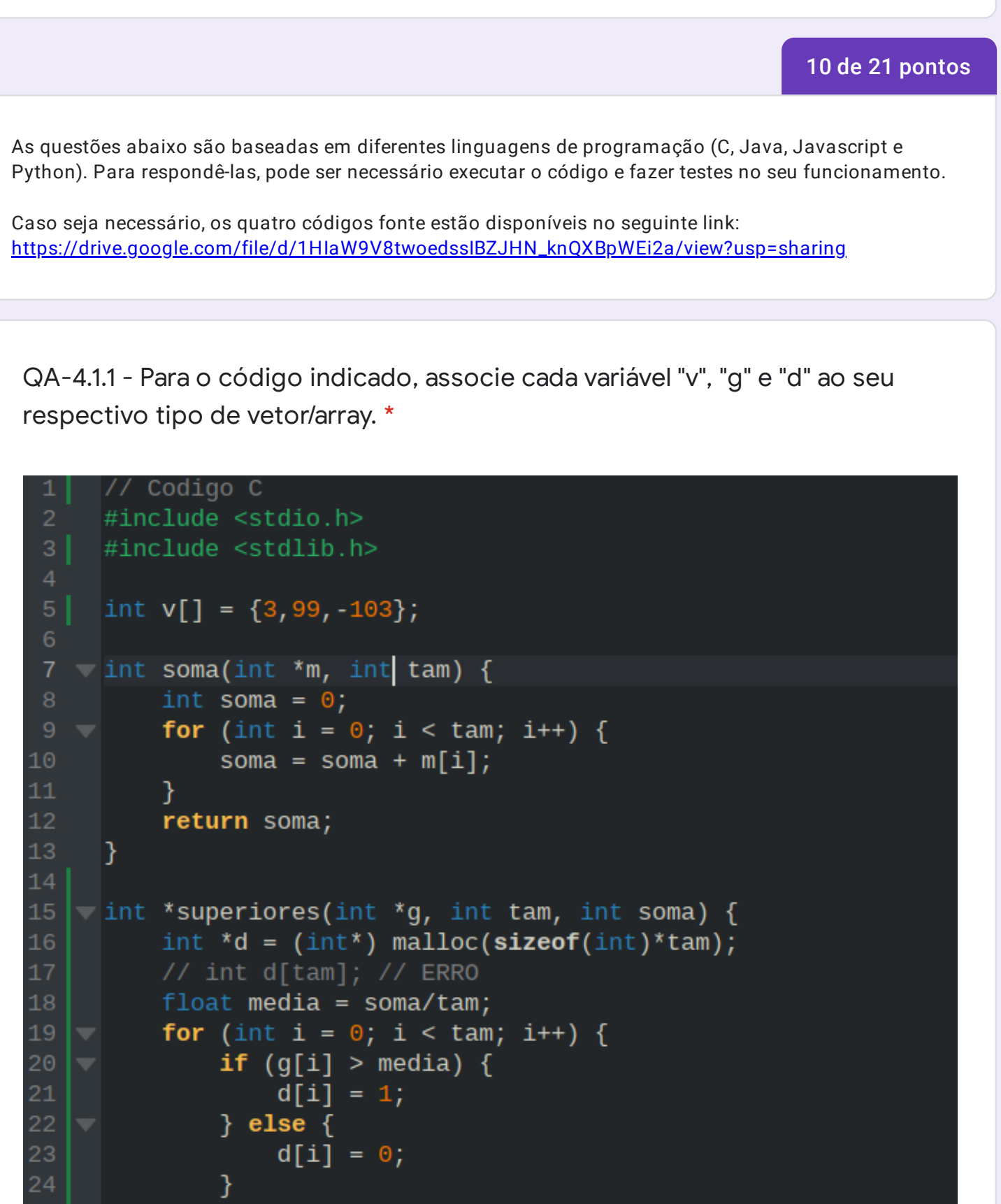
☐ ANDRE LUIZ DE PAULA MACHADO
☐ ANDRE PARANHOS CHAUD
☐ ANDRE TONI FOGUEL
☐ BRUNO GOMES
☐ DANILO BORGES DUTRA
☐ EDUARDO MELLO SAMPAIO BERNARDINO
☐ EZEQUIEL PIRES E SILVA
☐ FILIPE ANDRADE PERES
☐ GABRIEL ARRUDA PARANHOS NETTO
☐ GABRIELA FELIX DE SOUZA
☑ GIOVANNI DE OLIVEIRA FELISBERTO
☐ IGOR ALVES RIBEIRO
☐ ITALO MATOS OLIVEIRA
☐ JOAO PEDRO PORTILHO SILVA
☐ KAIO BORGES NASCIMENTO
☐ LUCAS ELIAS DE ANDRADE CRUVINEL
☐ RAFAEL DE CARVALHO
☐ RAMON SOARES MENDES DE MENESES LEITE

10 de 21 pontos

As questões abaixo são baseadas em diferentes linguagens de programação (C, Java, Javascript e Python). Para respondê-las, pode ser necessário executar o código e fazer testes no seu funcionamento.

Caso seja necessário, os quatro códigos fonte estão disponíveis no seguinte link:
https://drive.google.com/file/d/1HlaW9V8twoedssIBZJHN_knQXBpWEi2a/view?usp=sharing

QA-4.1.1 - Para o código indicado, associe cada variável "v", "g" e "d" ao seu respectivo tipo de vetor/array. *



```c
// Codigo C
#include <stdio.h>
#include <stdlib.h>

int v[] = {3,99,-103};

int soma(int *m, int tam) {
    int soma = 0;
    for (int i = 0; i < tam; i++) {
        soma = soma + m[i];
    }
    return soma;
}

int *superiores(int *g, int tam, int soma) {
    int *d = (int*) malloc(sizeof(int)*tam);
    // int d[tam]; // ERRO
    float media = soma/tam;
    for (int i = 0; i < tam; i++) {
        if (g[i] > media) {
            d[i] = 1;
        } else {
            d[i] = 0;
        }
    }
    return d;
}

void printmatriz(int *m, int tam) {
    printf("{");
    for (int i = 0; i < tam; i++) {
        printf("%i,",m[i]);
    }
    printf("}\n");
}

int main() {
    int somamatriz = soma(v,3);
    printf("%i\n",somamatriz);
    printmatriz(v,3);
    printmatriz(superiores(v,3,somamatriz),3);
}

```

	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.	Pontuação	
Vetor "v"	●	○	○	○	○	1/1	✓
Vetor "g"	○	●	○	○	○	1/1	✓
Vetor "d"	○	○	○	●	○	1/1	✓

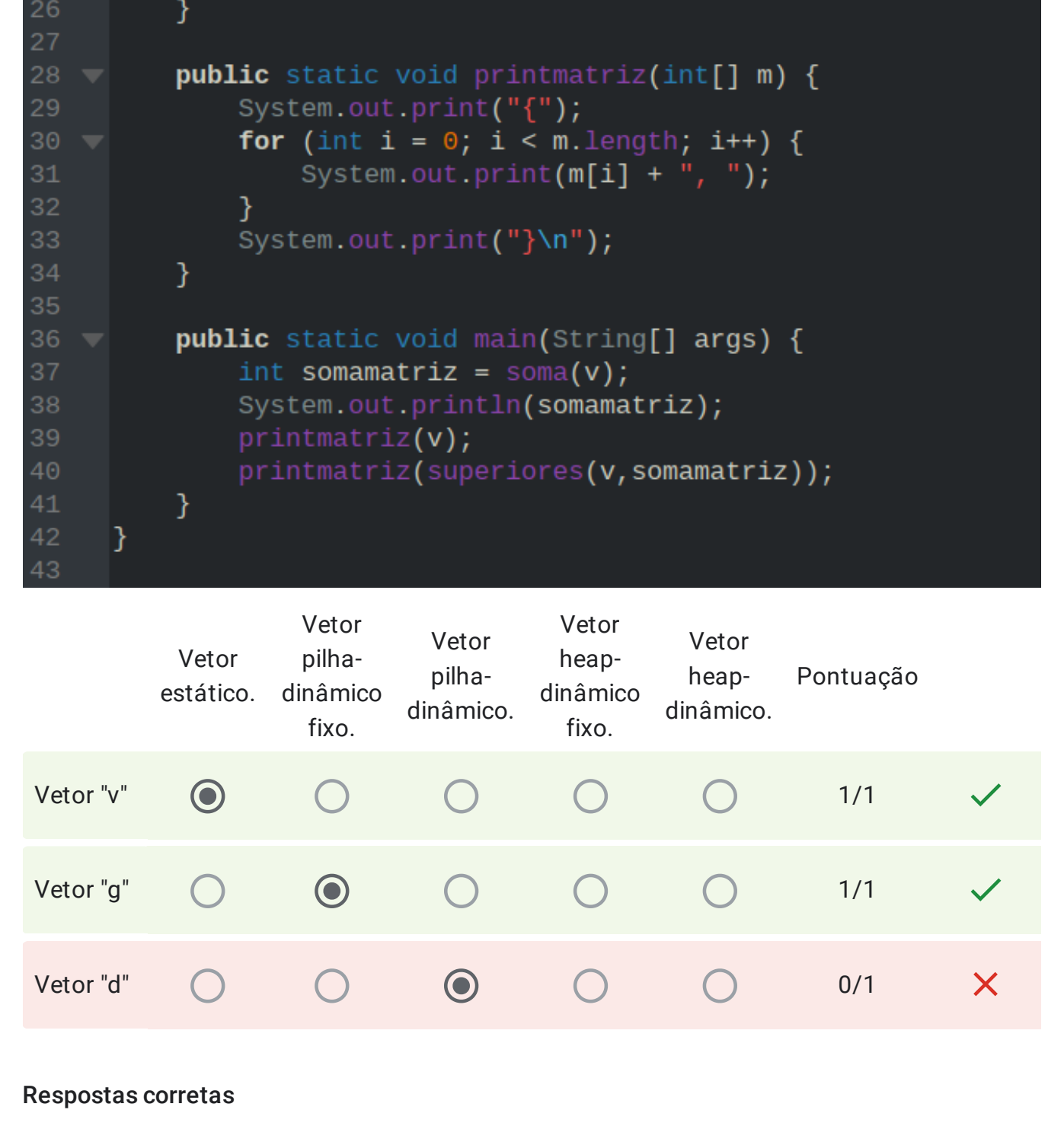
✗ QA-4.1.2 - Em quais desses vetores o compilador é capaz de definir a posição da memória em que um elemento ficará? 0/1

☑ Vetor "v" ✓
☐ Vetor "g"
☑ Vetor "d" ✗

Resposta correta

☑ Vetor "v"

QA-4.1.3 - Para o código indicado, associe cada variável "v", "g" e "d" ao seu respectivo tipo de vetor/array. *



```java
// Codigo Java

public class QaMatriz {

    public static int v[] = {3,99,-103};

    public static int soma(int[] m) {
        int soma = 0;
        for (int i = 0; i < m.length; i++) {
            soma = soma + m[i];
        }
        return soma;
    }

    public static int[] superiores(int []g, int soma) {
        int []d = new int[g.length];
        float media = soma/g.length;
        for (int i = 0; i < g.length; i++) {
            if (g[i] > media) {
                d[i] = 1;
            } else {
                d[i] = 0;
            }
        }
        return d;
    }

    public static void printmatriz(int[] m) {
        System.out.print("{");
        for (int i = 0; i < m.length; i++) {
            System.out.print(m[i] + ", ");
        }
        System.out.print("}\n");
    }

    public static void main(String[] args) {
        int somamatriz = soma(v);
        System.out.println(somamatriz);
        printmatriz(v);
        printmatriz(superiores(v,somamatriz));
    }
}

```

	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.	Pontuação	
Vetor "v"	●	○	○	○	○	1/1	✓
Vetor "g"	○	●	○	○	○	1/1	✓
Vetor "d"	○	○	●	○	○	0/1	✗

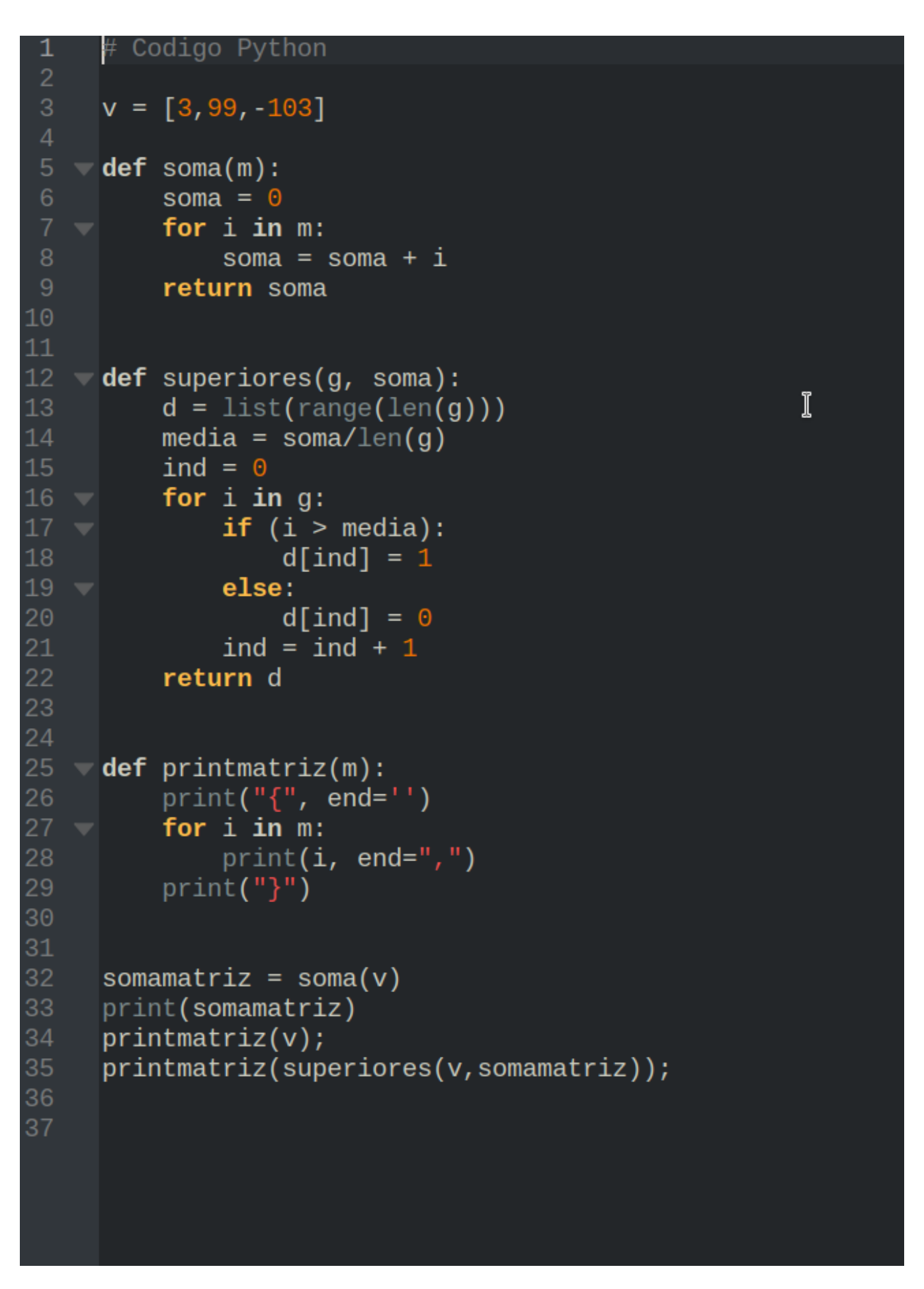
Respostas corretas

	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.
Vetor "d"	○	○	○	●	○

✓ QA-4.1.4 - Em quais desses vetores o compilador é capaz de definir a posição da memória em que um elemento ficará? 1/1

☑ Vetor "v" ✓
☐ Vetor "g"
☐ Vetor "d"

QA-4.1.5 - Para o código indicado, associe cada variável "v", "g" e "d" ao seu respectivo tipo de vetor/array. *



```python
# Codigo Python

v = [3,99,-103]

def soma(m):
    soma = 0
    for i in m:
        soma = soma + i
    return soma


def superiores(g, soma):
    d = list(range(len(g)))
    media = soma/len(g)
    ind = 0
    for i in g:
        if (i > media):
            d[ind] = 1
        else:
            d[ind] = 0
        ind = ind + 1
    return d


def printmatriz(m):
    print("{", end='')
    for i in m:
        print(i, end=",")
    print("}")


somamatriz = soma(v)
print(somamatriz)
printmatriz(v);
printmatriz(superiores(v,somamatriz));


```

	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.	Pontuação	
Vetor "v"	●	○	○	○	○	0/1	✗
Vetor "g"	○	●	○	○	○	0/1	✗
Vetor "d"	○	○	●	○	○	0/1	✗

Respostas corretas

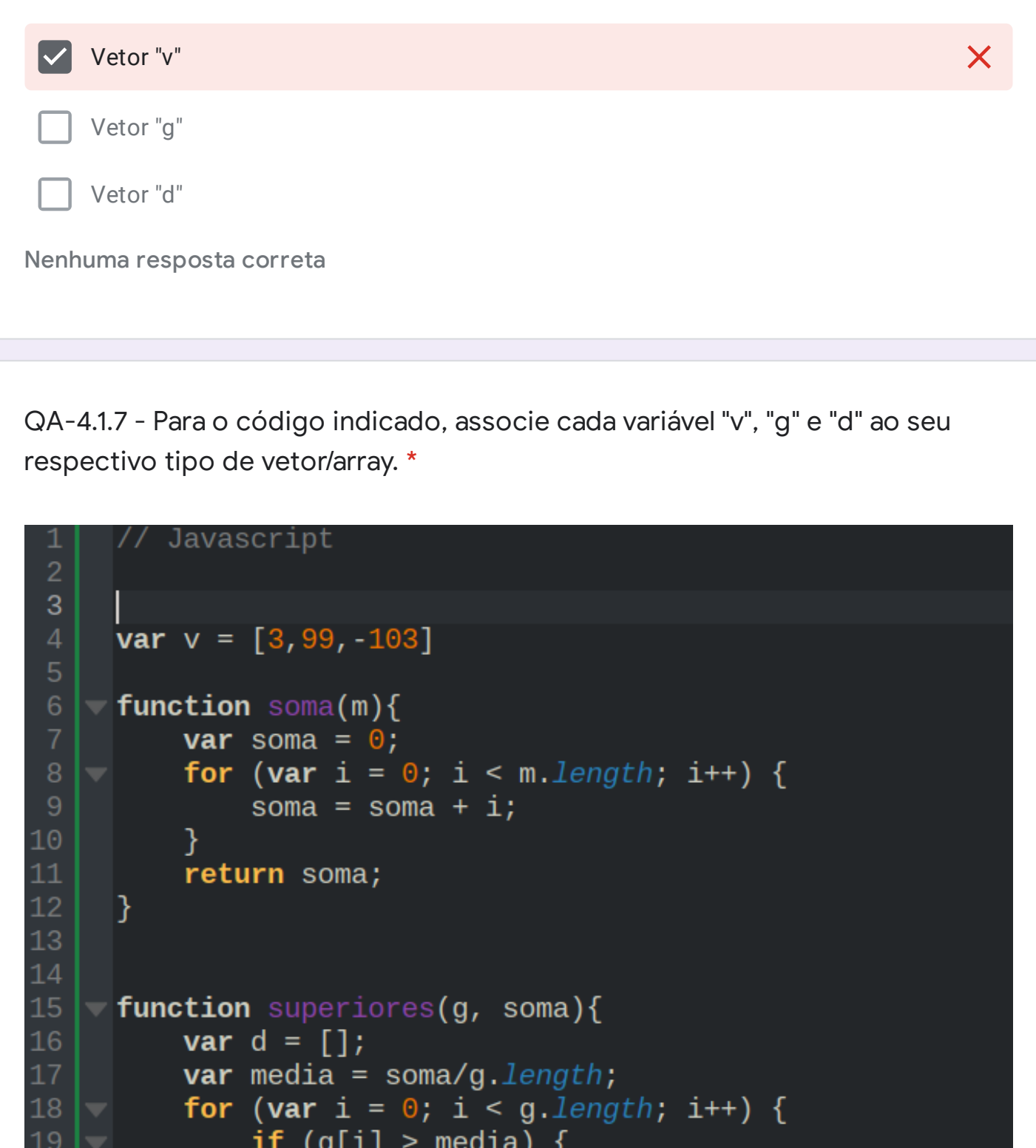
	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.
Vetor "v"	○	○	○	○	●
Vetor "g"	○	○	●	○	○
Vetor "d"	○	○	○	○	●

✗ QA-4.1.6 - Em quais desses vetores o compilador é capaz de definir a posição da memória em que um elemento ficará? .../1

☑ Vetor "v" ✗
☐ Vetor "g"
☐ Vetor "d"

Nenhuma resposta correta

QA-4.1.7 - Para o código indicado, associe cada variável "v", "g" e "d" ao seu respectivo tipo de vetor/array. *



```javascript
// Javascript


var v = [3,99,-103]

function soma(m){
    var soma = 0;
    for (var i = 0; i < m.length; i++) {
        soma = soma + i;
    }
    return soma;
}


function superiores(g, soma){
    var d = [];
    var media = soma/g.length;
    for (var i = 0; i < g.length; i++) {
        if (g[i] > media) {
            d[i] = 1;
        } else {
            d[i] = 0;
        }
    }
    return d;
}

function printmatriz(m) {
    var out = '{';
    for (var i = 0; i < m.length; i++) {
        out = out + m[i] + ', ';
    }
    console.log(out + '}');
}

var somamatriz = soma(v);
console.log(somamatriz);
printmatriz(v);
printmatriz(superiores(v,somamatriz));

```

	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.	Pontuação	
Vetor "v"	●	○	○	○	○	0/1	✗
Vetor "g"	○	●	○	○	○	0/1	✗
Vetor "d"	○	○	○	○	●	1/1	✓

Respostas corretas

	Vetor estático.	Vetor pilha-dinâmico fixo.	Vetor pilha-dinâmico.	Vetor heap-dinâmico fixo.	Vetor heap-dinâmico.
Vetor "v"	○	○	○	○	●
Vetor "g"	○	○	●	○	○

✗ QA-4.1.8 - Em quais desses vetores o compilador é capaz de definir a posição da memória em que um elemento ficará? .../1

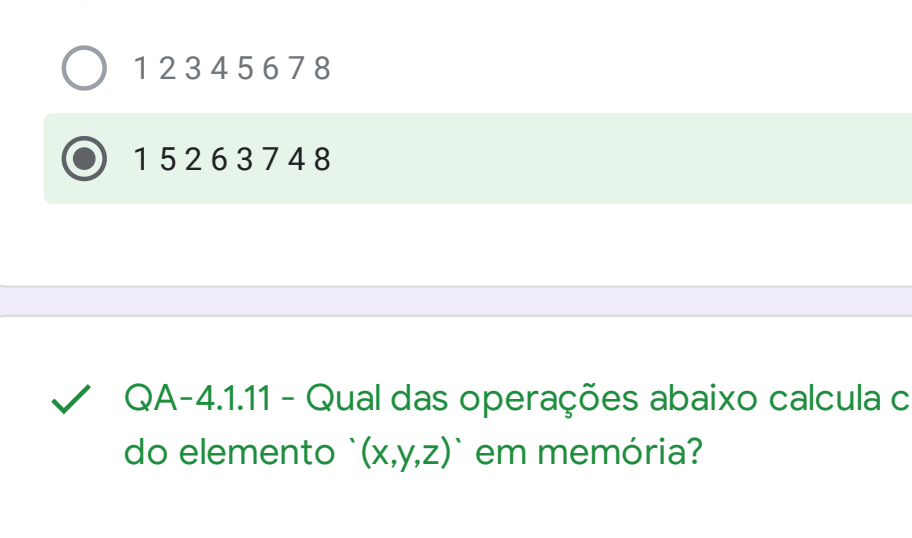
☑ Vetor "v" ✗
☐ Vetor "g"
☐ Vetor "d"

Nenhuma resposta correta

✓ QA-4.1.9 - Na linguagem C, qual é a diferença entre um vetor bidimensional de caracteres e um vetor de strings? 1/1

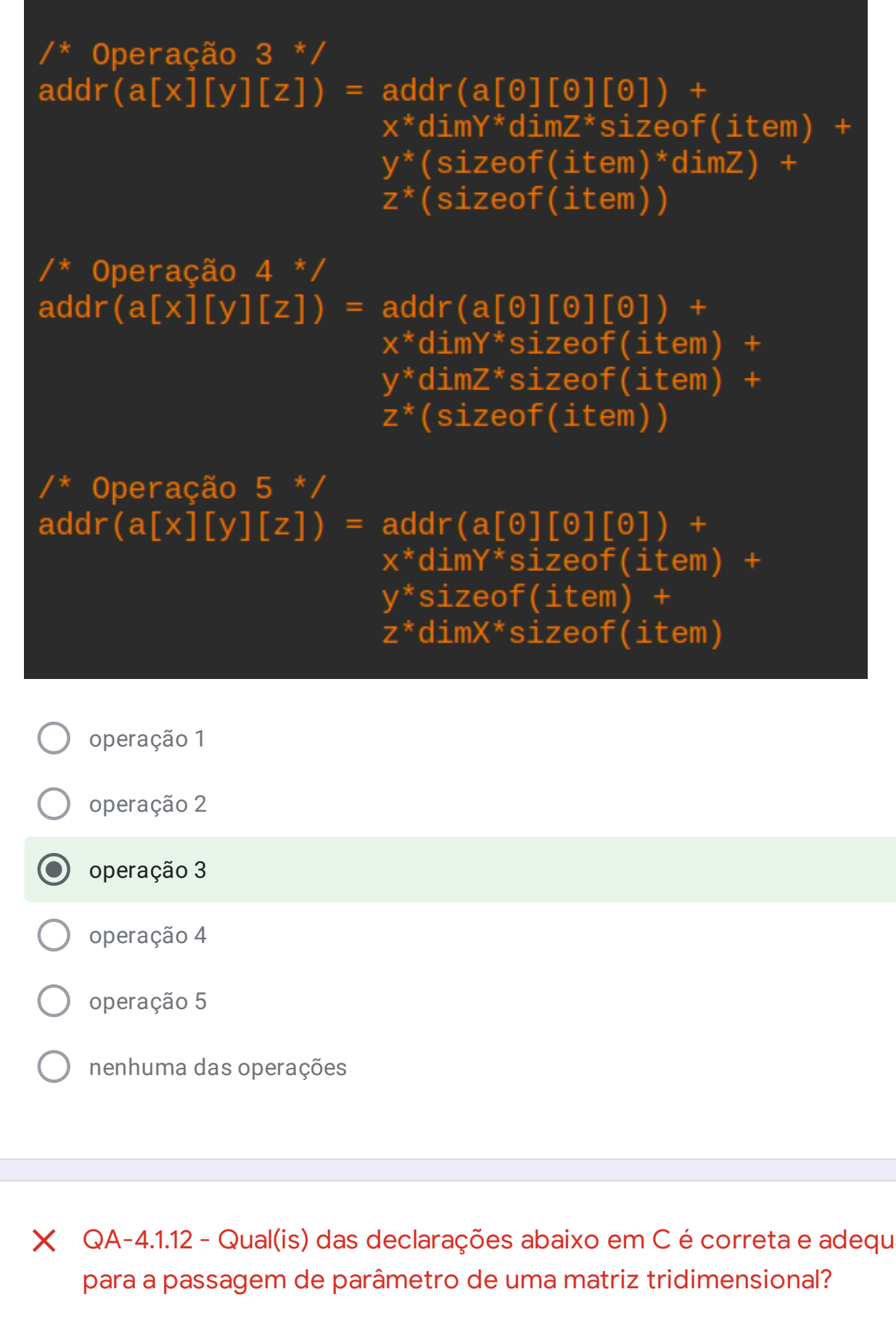
○ Facilidade do acesso aos elementos.
○ Vetor bidimensional não armazena strings terminadas em null, apenas caracteres.
● Vetor de strings armazena e aloca strings de tamanho variável, enquanto o vetor bidimensional aloca um tamanho fixo. ✓
○ Nenhuma

✓ QA-4.1.10 - Desenhe a distribuição em memória (sequência em que estão na memória) dos elementos da matriz 2x2x2 abaixo, considerando a implementação de matrizes de C. NOTA: o eixo "linha" está invertido na figura. 1/1



○ 1 3 2 4 5 7 6 8
○ 1 2 5 6 3 4 7 8
○ 1 2 3 4 5 6 7 8
● 1 5 2 6 3 7 4 8 ✓

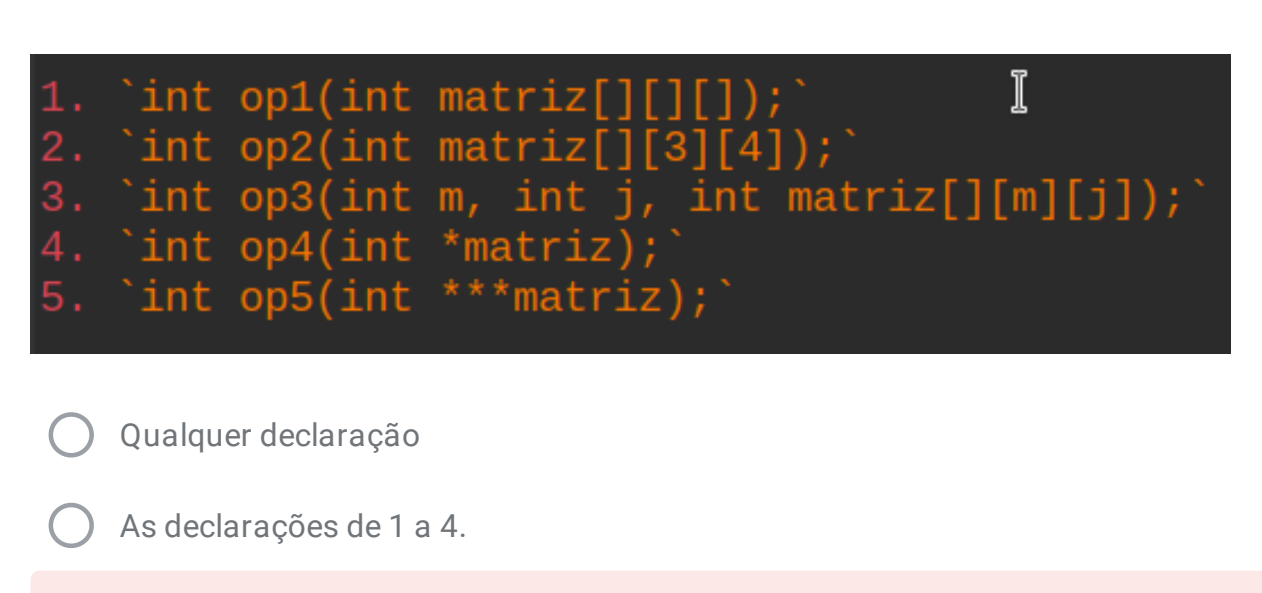
✓ QA-4.1.11 - Qual das operações abaixo calcula corretamente a posição do elemento `(x,y,z)` em memória? 1/1



```
/* Operação 1 */
addr(a[x][y][z]) = addr(a[0][0][0]) +
                   x*sizeof(item) +
                   y*(sizeof(item)*dimY) +
                   z*(sizeof(item)*dimZ)

/* Operação 2 */
addr(a[x][y][z]) = addr(a[0][0][0]) +
                   x*sizeof(item) +
                   y*dimZ*sizeof(item) +
                   z*dimX*sizeof(item)

/* Operação 3 */
addr(a[x][y][z]) = addr(a[0][0][0]) +
                   x*dimY*dimZ*sizeof(item) +
                   y*(sizeof(item)*dimZ) +
                   z*(sizeof(item))

/* Operação 4 */
addr(a[x][y][z]) = addr(a[0][0][0]) +
                   x*dimY*sizeof(item) +
                   y*dimZ*sizeof(item) +
                   z*(sizeof(item))

/* Operação 5 */
addr(a[x][y][z]) = addr(a[0][0][0]) +
                   x*dimY*sizeof(item) +
                   y*sizeof(item) +
                   z*dimX*sizeof(item)
```

○ operação 1
○ operação 2
● operação 3 ✓
○ operação 4
○ operação 5
○ nenhuma das operações

✗ QA-4.1.12 - Qual(is) das declarações abaixo em C é correta e adequada para a passagem de parâmetro de uma matriz tridimensional? 0/1



```
1. `int op1(int matriz[][][]);`
2. `int op2(int matriz[][3][4]);`
3. `int op3(int m, int j, int matriz[][m][j]);`
4. `int op4(int *matriz);`
5. `int op5(int ***matriz);`
```

○ Qualquer declaração
○ As declarações de 1 a 4.
● As declarações de 2 a 4. ✗
○ As declarações 2 e 3, sendo a mais adequada a 3.
○ Apenas a declaração 2.
○ Apenas a declaração 3.

Resposta correta

● As declarações 2 e 3, sendo a mais adequada a 3.

✗ QA-4.1.14 - Explique a sua resposta no item anterior. .../1

2- Está correta pois na passagem de matrizes em C, só não precisa especificar o tamanho da primeira dimensão.
3- Neste caso é basicamente a mesma coisa da declaração 2.
4- Já neste caso, é passado a referência, então a matriz consegue ser identificada.

Feedback

Apenas nessas duas opções (2 e 3), o compilador é capaz de calcular a posição dos elementos em cada uma das posições do vetor, a partir dos parâmetros passados. A opção 3 é mais adequada pois é mais flexível: é possível passar um vetor de diversas dimensões.

Mesmo que em outras o código compile (ao menos a assinatura da função), elas não podem ser utilizadas.